

Alpha–Beta Pruning and Depth-Limited Game Search

Lecture 9

CS 4880

Kirk Duran

Deep Blue: Search Is Powerful, but Expensive

Deep Blue's success depended on searching enormous game trees, but exhaustive minimax is wasteful.

- ▶ Many legal moves are strategically irrelevant.
- ▶ Some branches can be proven unable to change the final decision.
- ▶ In chess, terminal states may be far beyond any practical search horizon.

Two questions for this lecture

Can we **avoid searching irrelevant branches**, and can we **stop before terminal states** without giving up useful decisions?

Visual placeholder: Deep Blue / chess search tree with a small highlighted region and many faded branches.

Key idea

Practical game playing combines exact pruning with controlled approximations of search depth.

Minimax Review: Start at the Leaves

Recall a depth-3 minimax tree with alternating MAX and MIN layers.

- ▶ Terminal utilities are measured from MAX's perspective.
- ▶ MAX nodes choose the largest child value.
- ▶ MIN nodes choose the smallest child value.
- ▶ Values are propagated from leaves back toward the root.

Example tree

Leaf utilities, left to right: $-1, 3, 5, 1, -6, -4, 0, 9$.

Example placeholder: full binary minimax tree before any leaf has been evaluated. Highlight the first leaf, -1 .

Minimax Review: First Leaf

The leftmost MAX node begins by evaluating its first child.

- ▶ First observed utility: -1 .
- ▶ MAX's current best value is therefore -1 .
- ▶ The sibling must still be inspected because it may be larger.

Key idea

A partial search gives a bound on what a node can currently guarantee, but not necessarily its final minimax value.

Example placeholder: same tree with leaf -1 revealed and the leftmost MAX node temporarily valued at -1 .

Minimax Review: Finish the First MAX Node

The sibling leaf has utility 3.

$$\max(-1, 3) = 3.$$

- ▶ The leftmost MAX node returns 3 to its parent MIN node.
- ▶ This establishes one candidate outcome for that MIN node.

What MIN knows

MIN can already force an outcome no greater than 3 if it chooses this child.

*Example placeholder: leaves -1 and 3 revealed;
their MAX parent labeled 3 .*

Minimax Review: Finish the Left MIN Subtree

The second MAX child evaluates to

$$\max(5, 1) = 5.$$

The parent MIN node therefore returns

$$\min(3, 5) = 3.$$

- ▶ MAX would prefer the branch worth 5.
- ▶ MIN rejects it because the branch worth 3 is better for MIN.

Example placeholder: complete left half of tree, with MAX values 3 and 5, then MIN value 3.

Key idea

Minimax values encode what rational players can force, not what either player merely hopes will happen.

Minimax Review: Full Backed-Up Value

Completing the right subtree gives

$$\max(-6, -4) = -4, \quad \max(0, 9) = 9,$$

and therefore

$$\min(-4, 9) = -4.$$

At the root,

$$\max(3, -4) = 3.$$

Minimax decision

MAX chooses the left action because it guarantees utility 3 under optimal opposition.

Example placeholder: complete tree with internal values 3, 5, -4, 9, MIN values 3, -4, and root value 3.

Why Full Minimax Becomes Impractical

Even a few plies of chess create a rapidly expanding set of positions.

- ▶ Each legal move creates a new state.
- ▶ The opponent then has many replies.
- ▶ Alternating choices multiply the number of candidate continuations.
- ▶ Most of those continuations will never occur in actual play.

Search-tree growth

With branching factor b and depth m , exhaustive minimax examines $O(b^m)$ nodes.

Visual placeholder: four successive chess positions showing one line of play while many alternatives branch away.

The Cost of Looking Only Four Rounds Ahead

The source example expands a chess tree only a few rounds deep and already produces an enormous number of nodes.

- ▶ Brute-force minimax treats every legal continuation as potentially relevant.
- ▶ Yet many branches are dominated by alternatives already discovered elsewhere.
- ▶ If a branch cannot change the root decision, evaluating it is wasted work.

Optimization target

Preserve the exact minimax result while examining fewer nodes.

Visual placeholder: wide chess minimax tree after four rounds; annotate the source example's roughly 206,000 explored nodes.

Pruning Intuition: A Branch Already Looks Too Good for MAX

Return to the same minimax tree.

- ▶ The first MAX child under the left MIN node has value 3.
- ▶ In the sibling MAX node, the first examined leaf has value 5.
- ▶ Therefore that MAX node will return at least 5, regardless of its unseen sibling.

Observation

MIN already has a child worth 3. Why would MIN ever choose a sibling guaranteed to be at least 5?

Example placeholder: left MIN subtree with known child value 3 and sibling MAX node currently known to be ≥ 5 ; hide the remaining leaf.

Pruning Intuition: MIN Will Never Choose That Branch

MIN prefers smaller values.

- ▶ MIN can already choose the branch worth 3.
- ▶ The competing MAX branch is guaranteed to be at least 5.
- ▶ Its unseen child cannot make that branch better for MIN.

Safe conclusion

The remaining child of that MAX node can be ignored without changing the minimax value of the parent.

Example placeholder: same tree with the unknown sibling visibly crossed out or faded; left MIN value remains 3.

Pruning Intuition: The Symmetric Case

The same reasoning appears on the right side from MAX's perspective.

- ▶ The root already has a left action worth 3.
- ▶ Suppose the right MIN node discovers a child worth -4 .
- ▶ That MIN node can force a value no greater than -4 .
- ▶ MAX would never choose that right action over the known value 3.

Example placeholder: root known to have value at least 3 from its left child; right MIN subtree known to be ≤ -4 with an unexplored sibling.

Key idea

Once an ancestor already has a better alternative, some descendants become irrelevant to the final decision.

Pruning Preserves the Same Root Decision

We have now skipped two portions of the tree.

- ▶ The pruned leaves are not approximations.
- ▶ Their exact values are unnecessary because they cannot affect the backed-up root value.
- ▶ The root still evaluates to 3.

Goal achieved

Search fewer nodes while returning exactly the same move as full minimax.

Example placeholder: completed root value 3 with pruned leaves faded; emphasize identical decision to full minimax.

How Much Can Pruning Save?

In the source demonstration, alpha-beta search at depth 4 explored only a tiny fraction of the positions examined by naive minimax.

- ▶ Minimax: roughly 206,000 positions in the illustrated tree.
- ▶ Alpha-beta: roughly 580 positions in the illustrated comparison.
- ▶ Reported search fraction: about 0.28% of the brute-force total.

Important qualification

The savings depend strongly on the order in which moves are examined.

Visual placeholder: side-by-side search trees labeled "Minimax" and "Alpha-Beta," matching the source comparison.

Alpha-Beta Pruning

Definition

Alpha-beta pruning computes the same minimax value as exhaustive minimax while avoiding branches that cannot affect the final decision.

The search maintains two bounds along the current path:

- ▶ α : the best value MAX can guarantee so far.
- ▶ β : the best value MIN can guarantee so far.

Cutoff condition

When $\alpha \geq \beta$, the remaining children of the current node cannot influence the final minimax choice and may be pruned.

Visual placeholder: game tree with α and β bounds along the path and scissors cutting an irrelevant subtree.

Key idea

Alpha and beta summarize what the ancestors already know well enough to reject some unexplored alternatives.

Alpha-Beta Search Algorithm

Recursive structure

```
ALPHA_BETA(state,  $\alpha$ ,  $\beta$ , maximizing)
    if TERMINAL(state): return UTILITY(state)
    if maximizing:
        value  $\leftarrow -\infty$ 
        for action in ACTIONS(state):
            value  $\leftarrow \max(\text{value}, \text{ALPHA\_BETA}(\text{RESULT}(\dots), \alpha, \beta, \text{false}))$ 
             $\alpha \leftarrow \max(\alpha, \text{value})$ 
            if  $\alpha \geq \beta$ : break
        return value
    else:
        value  $\leftarrow +\infty$ 
        for action in ACTIONS(state):
            value  $\leftarrow \min(\text{value}, \text{ALPHA\_BETA}(\text{RESULT}(\dots), \alpha, \beta, \text{true}))$ 
             $\beta \leftarrow \min(\beta, \text{value})$ 
            if  $\alpha \geq \beta$ : break
        return value
```

- ▶ Same recursive minimax structure.
- ▶ Pass α and β downward.
- ▶ Update the relevant bound after each child.
- ▶ Stop when the search window closes.

Code screenshot placeholder: Python alpha-beta implementation.

Alpha-Beta Example: Initialize the Search

Start at the root MAX node with

$$\alpha = -\infty, \quad \beta = +\infty.$$

These bounds are inherited by descendants until tighter information is discovered.

First step

Descend depth-first to the leftmost terminal leaf, whose utility is -1 .

Trace placeholder: initial alpha-beta tree; show $\alpha = -\infty, \beta = +\infty$ at root and highlight leaf -1 .

Alpha-Beta Example: Update α at MAX

At the leftmost MAX node, the first child returns -1 .

$$\alpha \leftarrow \max(-\infty, -1) = -1.$$

- ▶ MAX can now guarantee at least -1 at this node.
- ▶ The sibling is still relevant because it may improve MAX's value.

*Trace placeholder: leftmost MAX node annotated
 $\alpha = -1, \beta = +\infty$ after leaf -1 .*

Alpha-Beta Example: Finish the First MAX Node

The sibling leaf returns 3.

$$\alpha \leftarrow \max(-1, 3) = 3.$$

The MAX node is now complete and returns 3 to its parent MIN node.

No cutoff yet

$\alpha = 3$ and $\beta = +\infty$, so the current window remains open.

Trace placeholder: leaves $-1, 3$ evaluated; first MAX node labeled value 3 and $\alpha = 3$.

Alpha-Beta Example: Update β at MIN

The parent is a MIN node. Its first child returns 3.

$$\beta \leftarrow \min(+\infty, 3) = 3.$$

- ▶ MIN can now guarantee a value no greater than 3.
- ▶ This $\beta = 3$ bound is passed to the next MAX child.

*Trace placeholder: left MIN node labeled
 $\alpha = -\infty, \beta = 3$; move to its second MAX child.*

Alpha-Beta Example: A Candidate Already Exceeds β

The second MAX child is searched under the window

$$\alpha = -\infty, \quad \beta = 3.$$

Its first leaf has value 5, so

$$\alpha \leftarrow 5.$$

Critical comparison

Now $\alpha = 5 \geq \beta = 3$.

Trace placeholder: second MAX node with first leaf 5 revealed; annotate $\alpha = 5, \beta = 3$.

Alpha-Beta Example: First Cutoff

Because $\alpha \geq \beta$, the remaining child is irrelevant.

- ▶ This MAX node is already worth at least 5.
- ▶ Its parent MIN node already has an alternative worth 3.
- ▶ MIN will never select a branch that cannot fall below 5.

Prune

Do not evaluate the sibling leaf 1.

*Trace placeholder: show cutoff at $\alpha = 5, \beta = 3$
and cross out the unseen leaf 1.*

Key idea

The cutoff is justified by ancestor preferences, not by guessing the value of the pruned node.

Alpha-Beta Example: Return the Left Subtree Value

The left MIN node compares its children:

- ▶ first child: exactly 3;
- ▶ second child: known to be at least 5.

Therefore MIN returns 3.

$$\min(3, \geq 5) = 3.$$

At the root

MAX updates its bound to $\alpha = 3$.

*Trace placeholder: left MIN subtree returns 3;
pruned leaf remains faded. Root now records
 $\alpha = 3$.*

Alpha-Beta Example: Search the Right MIN Subtree

The root enters its right MIN child with

$$\alpha = 3, \quad \beta = +\infty.$$

The first MAX grandchild evaluates leaves -6 and -4 :

$$\max(-6, -4) = -4.$$

This value is returned to the right MIN node.

*Trace placeholder: right MIN subtree active; first
MAX child evaluates $-6, -4$ and returns -4
while root bound $\alpha = 3$ is visible.*

Alpha-Beta Example: Update β to -4

At the right MIN node,

$$\beta \leftarrow \min(+\infty, -4) = -4.$$

But this subtree was entered with $\alpha = 3$ from the root.

Window closes

$$\alpha = 3 \geq \beta = -4.$$

MIN can already force -4 , which is worse for MAX than the root's known alternative 3 .

*Trace placeholder: right MIN node labeled
 $\alpha = 3, \beta = -4$ after its first child returns -4 .*

Alpha-Beta Example: Second Cutoff and Final Decision

The rest of the right MIN subtree can be pruned.

- ▶ The unseen MAX child containing 0 and 9 is never evaluated.
- ▶ The right root action is known to be no better than -4 .
- ▶ The root already has an action worth 3.

Final result

Alpha-beta returns root value 3, exactly matching full minimax.

Trace placeholder: final alpha-beta tree with both cutoffs visible and root value 3.

The Pruning Logic

At a MAX node

- ▶ MAX searches for the highest value.
- ▶ α tracks MAX's best guarantee so far.
- ▶ If $\alpha \geq \beta$, an ancestor MIN node already has a better alternative.

At a MIN node

- ▶ MIN searches for the lowest value.
- ▶ β tracks MIN's best guarantee so far.
- ▶ If $\beta \leq \alpha$, an ancestor MAX node already has a better alternative.

Visual placeholder: compact annotated tree showing one MAX cutoff and one MIN cutoff.

Key idea

A cutoff occurs when the current branch falls outside the range of values that could still matter to an ancestor.

Alpha–Beta Properties

Property	Alpha–beta search
Complete	Yes, for finite game trees.
Optimal	Yes, under the same optimal-play assumptions as minimax.
Worst-case time	$O(b^m)$, the same asymptotic bound as minimax.
Best-case time	$O(b^{m/2})$ with near-perfect move ordering.
Space	$O(m)$ for depth-first recursive generation, ignoring stored move lists.

Key idea

Alpha–beta improves efficiency without changing the minimax decision; move ordering determines how much pruning actually occurs.

Pruning Is Not Enough

Even after alpha–beta pruning, chess trees remain extremely deep.

- ▶ A game may require dozens of moves before reaching checkmate, resignation, or draw.
- ▶ Some variations are much deeper than others.
- ▶ Waiting for terminal utilities is therefore unrealistic under fixed time limits.

New problem

If search stops before the game ends, what value should be assigned to a nonterminal position?

*Visual placeholder: Deep Blue / chessboard
with an uneven deep tree extending far
below the current search frontier.*

Depth-Limited Minimax

Definition

Depth-limited minimax stops search at a cutoff depth and evaluates the resulting nonterminal states with an evaluation function.

Replace the terminal-only base case with two possibilities:

- ▶ if the state is terminal, return its true utility;
- ▶ if the cutoff condition is reached, return $EVAL(s)$.

The returned estimates are then backed up through MAX and MIN exactly as before.

Visual placeholder: game tree truncated by a horizontal "search horizon"; frontier nodes labeled with heuristic values.

Key idea

Depth limiting trades exact terminal reasoning for a heuristic estimate of positions near the search horizon.

Depth Limits and Pruning Work Together

Practical game search usually combines both mechanisms.

- ▶ **Depth limit:** controls how far the tree may grow.
- ▶ **Alpha-beta:** avoids searching branches within that horizon that cannot matter.
- ▶ **Evaluation function:** estimates frontier states when terminal utilities are unavailable.

Source comparison

The lecture's depth-4 example reports alpha-beta examining about 0.28% of the nodes shown for brute-force minimax.

Visual placeholder: repeat source side-by-side minimax vs. alpha-beta depth-4 trees, now annotate the shared cutoff depth.

Depth-Limited Alpha–Beta Search

Algorithmic structure

```
ALPHA_BETA(state, depth,  $\alpha$ ,  $\beta$ , maximizing)
  if TERMINAL(state): return UTILITY(state)
  if CUTOFF(state, depth): return EVAL(state)
  if maximizing:
    value  $\leftarrow -\infty$ 
    for action in ACTIONS(state):
      value  $\leftarrow \max(\text{value}, \text{ALPHA\_BETA}(\text{RESULT}(\dots), \text{depth}+1, \alpha, \beta, \text{false}))$ 
       $\alpha \leftarrow \max(\alpha, \text{value})$ 
      if  $\alpha \geq \beta$ : break
    return value
  else: symmetrically minimize and update  $\beta$ 
```

- ▶ Track current depth.
- ▶ Stop at terminal or cutoff states.
- ▶ Evaluate frontier states heuristically.
- ▶ Preserve alpha–beta pruning above the frontier.

*Code screenshot placeholder:
depth-limited alpha–beta.*

Evaluation Functions Replace Unreachable Utilities

When search stops before terminal states, the agent needs a proxy for true utility.

- ▶ The evaluation should correlate with the probability or quality of eventual outcomes.
- ▶ Positive values should favor MAX; negative values should favor MIN.
- ▶ Evaluation must be cheap enough to apply at many frontier nodes.

Design objective

Estimate the strategic value of a state using features available now, without solving the remainder of the game.

Visual placeholder: Deep Blue examining a frontier of chess states, each assigned a heuristic score.

Chess Evaluation Functions

A classical evaluation function combines weighted strategic features:

$$\text{EVAL}(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s).$$

Typical features include

- ▶ material balance;
- ▶ king safety;
- ▶ pawn structure;
- ▶ mobility and control of important squares;
- ▶ piece activity and positional constraints.

Example placeholder: chess position annotated with material, king-safety, pawn-structure, and mobility features.

Key idea

An evaluation function is useful only insofar as its features predict long-term game outcomes beyond the current search depth.

Connect Four: A Simple Heuristic Evaluation

A Connect Four heuristic can score patterns that indicate progress toward a win.

- ▶ Large positive score for four in a row.
- ▶ Smaller positive score for three in a row with an open cell.
- ▶ Smaller still for two in a row with multiple open cells.
- ▶ Apply corresponding negative values to dangerous opponent patterns.

Example scoring sketch

+1000 for four; +100 for an open three; +10 for an open two, with analogous penalties for the opponent.

Example placeholder: Connect Four board with highlighted windows/patterns and a small scoring-function code excerpt.

The Horizon Effect

Definition

The **horizon effect** occurs when important consequences lie just beyond the search cutoff, causing the evaluation function to misjudge the current position.

- ▶ A tactic may look favorable only because its refutation is one ply beyond the horizon.
- ▶ An agent may delay an unavoidable loss until after the cutoff.
- ▶ The frontier position appears stable even though a forcing sequence is imminent.

Consequence

A deeper fixed cutoff helps, but does not eliminate the problem; the tactical event can simply move beyond the new horizon.

Visual placeholder: horizon line with a move that appears strong before the line but loses immediately beyond it.

Stable vs. Unstable Frontier Positions

Quiescent position

A strategically stable position with no immediate forcing tactical sequence.

- ▶ no urgent captures;
- ▶ no immediate checks;
- ▶ no major promotions or exchanges pending.

Non-quiescent position

An unstable position in which the evaluation may change sharply after a few forcing moves.

- ▶ tactical captures;
- ▶ checks or threats;
- ▶ promotions;
- ▶ unresolved material swings.

Key idea

Evaluation is most trustworthy when the cutoff occurs in a relatively stable position.

Quiescence Search

Definition

Quiescence search selectively extends search beyond the nominal depth limit when the frontier position is tactically unstable.

Rather than considering every legal move, continue only through forcing moves such as

- ▶ captures;
- ▶ checks;
- ▶ promotions;
- ▶ other domain-specific tactical actions.

Stop once a sufficiently stable state is reached and then apply the evaluation function.

Visual placeholder: main alpha-beta tree ending at a horizon, with a few selectively extended tactical branches below it.

Key idea

Quiescence search spends extra depth where the evaluation function is least reliable.

Singular Extensions

A **singular extension** searches deeper along a line when one move appears substantially stronger than all alternatives.

- ▶ Trigger when the best move is clearly separated from the rest.
- ▶ Allocate additional depth to that critical continuation.
- ▶ Avoid spending equal depth on obviously inferior alternatives.

Why it helps

Some strategically decisive sequences are narrow rather than broad; selective depth can reveal their true consequences.

Visual placeholder: tree with one brightly highlighted branch extended several plies deeper than its siblings.

Forward Pruning: Search Less by Taking Risk

Forward pruning deliberately discards or reduces branches before their irrelevance is proven.

- ▶ Reduces search effort aggressively.
- ▶ Can reach greater effective depth.
- ▶ Sacrifices the minimax guarantee if a discarded move was actually important.

Examples from the source

- ▶ Beam-style move selection.
- ▶ ProbCut / probability-based pruning.
- ▶ Late-move reductions (LMR).

Visual placeholder: only a few promising branches remain bright while others are intentionally discarded.

Key idea

Alpha-beta pruning is exact; forward-pruning methods are selective approximations that trade certainty for speed.

Search vs. Lookup

Opening knowledge

Familiar early-game positions can be stored in opening books rather than rediscovered through search every game.

- ▶ curated or statistically derived lines;
- ▶ immediate access to known strong moves;
- ▶ saves computation for unfamiliar positions.

Endgame knowledge

Small endgames can be solved exhaustively in advance and stored in tablebases.

- ▶ exact win/draw/loss information;
- ▶ often exact distance-to-conversion information;
- ▶ replace online search with direct lookup when applicable.

Visual placeholder: opening book on the left, endgame tablebase/database on the right, search in the middle.

Iterative Deepening Under a Time Limit

Definition

Iterative deepening repeatedly performs depth-limited search with increasing depth until the available time is exhausted.

1. Search to depth 1.
2. Repeat at depths 2, 3, ...
3. Stop when the time budget is nearly exhausted.
4. Return the best move from the deepest fully completed iteration.

Why repeat work?

Earlier iterations improve move ordering for deeper alpha-beta searches and guarantee an anytime answer if computation must stop unexpectedly.

Visual placeholder: nested depth-1, depth-2, depth-3 search horizons around a chess clock.

Deep Blue as a Search System

The source lecture uses Deep Blue to connect the mechanisms into one practical architecture:

- ▶ minimax-style adversarial reasoning;
- ▶ alpha-beta pruning;
- ▶ depth-limited search;
- ▶ heuristic evaluation of nonterminal states;
- ▶ quiescence search and selective extensions;
- ▶ forward-pruning ideas;
- ▶ opening books and endgame knowledge;
- ▶ progressive / iterative deepening.

Visual placeholder: Deep Blue match photo plus a layered pipeline from search to pruning to evaluation to lookup.

Key idea

Strong game-playing systems are not one algorithm; they are carefully engineered combinations of exact search, heuristic evaluation, selective depth, and stored knowledge.

Two Modern Paths to Strong Chess AI

Search-engine tradition

Systems such as Stockfish build on highly optimized tree search.

- ▶ alpha–beta-style pruning;
- ▶ transposition tables;
- ▶ selective reductions and extensions;
- ▶ strong evaluation functions;
- ▶ opening and endgame knowledge.

Learning-centered tradition

Systems such as AlphaZero combine learned value/policy models with Monte Carlo Tree Search.

- ▶ self-play learning;
- ▶ learned state evaluation;
- ▶ policy-guided search;
- ▶ no classical alpha–beta requirement.

Key idea

The core problem remains selective lookahead: devote computation to the continuations most likely to matter.

Chess AI: Comparing Machine and Human Strength

The source slide compares top human ratings with contemporary engine ratings to emphasize the scale of modern chess-engine performance.

- ▶ Engine strength now substantially exceeds elite human play under standard engine-testing conditions.
- ▶ Exact numerical rankings change over time and depend on the rating pool and hardware configuration.
- ▶ For lecture use, the conceptual point is more durable than any one snapshot.

Update placeholder

Replace with a current human-vs.-engine rating snapshot before presenting if numerical rankings matter.

Visual placeholder: current top-human rating list beside a current computer-chess engine rating list.

Real Games Violate the Idealized Model

Assumptions so far

- ▶ opponent plays optimally;
- ▶ legal actions are known;
- ▶ state is fully observable;
- ▶ action outcomes are deterministic;
- ▶ search can model the relevant future.

Complications in real games

- ▶ opponents can be suboptimal or adaptive;
- ▶ chance may affect outcomes;
- ▶ information may be hidden;
- ▶ exact outcomes may be probabilistic;
- ▶ the full tree may be too large even for sophisticated pruning.

Key idea

Alpha-beta is extremely effective for deterministic adversarial trees, but other game models require different search machinery.

Takeaways and Next Time

- ▶ Alpha–beta pruning returns the same move as minimax while skipping branches that cannot affect the decision.
- ▶ α records MAX's best guarantee; β records MIN's best guarantee; prune when $\alpha \geq \beta$.
- ▶ Move ordering controls how much pruning occurs: the best case reduces effective search from $O(b^m)$ toward $O(b^{m/2})$.
- ▶ Depth-limited search replaces unreachable terminal utilities with heuristic evaluation at a cutoff frontier.
- ▶ Horizon effects motivate quiescence search and selective extensions near tactically unstable positions.
- ▶ Iterative deepening provides time-bounded, anytime search while improving move ordering for alpha–beta.
- ▶ Next lecture: **Monte Carlo Tree Search**.

Key idea

Practical adversarial search succeeds by deciding both *which branches deserve exploration* and *how deeply each branch should be explored*.